Toward Agent-Driven Pathfinding for Heterogeneous LLM Serving Systems

Wonung Kim*, Hyunmin Choi*, Jongse Park
Korea Advanced Institute of Science and Technology
{wukim, hmchoi, jspark}@casys.kaist.ac.kr

Abstract

Efficient LLM serving increasingly relies on heterogeneous deployments that match each part of inference to hardware with suitable compute and memory characteristics, making their design a coupled hardware–software problem. Design space exploration (DSE) can handle such coupling by searching interacting decisions together, but only after experts formulate the problem by parameterizing the design and building an evaluator for it. This upfront effort pays off only when many candidates reuse the same formulation. In heterogeneous serving, however, exploration itself keeps introducing components and interactions that the formulation does not capture, so revising the formulation, rather than searching within it, becomes the bottleneck. To fold formulation into the search itself, we argue for *agent-driven pathfinding*, a DSE paradigm in which an AI agent charts the design space while exploring it. In this paradigm, the agent implements each hypothesis as an executable system modification and extends the evaluator whenever the modification falls outside it, letting exploration reach designs that no upfront formulation anticipated. Realizing this vision requires evaluation infrastructure that agents can operate and extend, experience that preserves evidence across reformulations, and human-in-the-loop control that keeps architects steering exploration. Because the evaluator now evolves with the search, its validity can no longer be assumed, leaving two open problems: search amplifies evaluator error, and each extension requires independent validation. We believe agent-driven pathfinding can reshape how the computer architecture and systems community conducts research by coupling human architectural judgment with agent-driven implementation and experimentation, bringing design questions that are impractical to explore today within reach.

1 Introduction

The cost of large language model (LLM) inference is becoming a first-order constraint on the growth of AI services. Reasoning models improve accuracy by spending additional computation at inference time [11, 31], and spending on cloud infrastructure for inference is projected to overtake spending for training in 2026 [10]. Serving efficiency now decides which models and applications are economically viable to deploy.

A promising path to higher efficiency is heterogeneous serving, which matches each part of inference to resources with suitable compute, memory, and cost characteristics. Recent systems separate prefill and decode [36, 52], independently scale attention and expert computation [53], offload memory-intensive operators to near-memory devices [34], and tier the KV cache across storage media [37]. Providers also operate fleets spanning hardware generations and accelerator vendors [3, 14, 28].

To realize these gains, however, a deployment must jointly optimize work and state placement, resource provisioning, and data movement [28, 52]. These decisions cannot be made in isolation because each changes the load, communication, and memory pressure seen by the other components. A locally efficient choice can move the bottleneck or add transfer overhead that erases its benefit.

Design space exploration (DSE) seems well suited to such coupled decisions because it searches them together. Conventional DSE frameworks [20, 22, 33], however, begin only after experts have encoded the relevant decisions as parameters and built an evaluator. This upfront effort is tractable when a stable formulation supports enough candidate designs to amortize its construction cost. Heterogeneous serving changes this balance because exploration increasingly varies the components and interactions that the formulation must capture. Separating prefill and decode introduces distinct resource pools and a KV-cache transfer path [36, 52], while moving attention to memory-side compute introduces device behavior absent from conventional serving models [13]. Each such structural change requires experts to revise the candidate representation and evaluator before search can resume. As heterogeneous deployments span more specialized devices and execution boundaries, these revisions recur more often and consume a growing share of the overall exploration effort.

We therefore argue that DSE for heterogeneous LLM serving must treat candidate construction and formulation revision as part of exploration rather than as prerequisites to it. A candidate is not merely a parameter assignment but an executable system modification paired with an evaluator that can measure it. Evidence from testing a candidate can refine the modification, revise the design space with newly exposed mechanisms and constraints, or extend the evaluator with missing models. We call this process *pathfinding* because the design space becomes an evolving output of exploration rather than a fixed input.

AI agents make this broader loop plausible because they can operate on the software artifacts that define both a design and its evaluator. Modern coding agents work in large repositories [15], modifying artifacts, invoking tools, and revising failed implementations [2, 32]. Agent-driven discovery systems use the same artifact-and-feedback loop to improve algorithms [30] and compiler heuristics [6] beyond expert-designed baselines. Applied to DSE, this loop can turn a hypothesis into an executable modification, evaluate it, and use the resulting evidence to determine the next design action. The agent complements conventional optimizers by constructing and revising formulations, while established optimizers solve the fixed subproblems that emerge within them.

*co-first authors

This position paper develops agent-driven pathfinding as a future DSE paradigm in which candidate construction, evaluator extension, and search form one evolving loop. Making this loop practical requires evaluation infrastructure that agents can operate and extend, experience that preserves evidence across reformulations, and human-in-the-loop control that keeps architects steering exploration. Because the evaluator now evolves with the search, its validity can no longer be assumed, leaving two open problems: search amplifies evaluator error, and each extension requires independent validation. We believe agent-driven pathfinding can reshape architecture and systems research by pairing architects' judgment with agents that implement and test their ideas, bringing within reach design questions that are impractical to explore today.

2 The Fixed-Formulation Bottleneck

2.1 What Conventional DSE Amortizes

Conventional DSE separates formulation from search. Before search begins, experts translate design choices into parameters, encode which combinations are legal as constraints, and build an evaluator that maps each legal assignment to performance, energy, and cost metrics [20, 22, 33]. During search, an optimizer generates legal assignments, invokes the evaluator on each, and ranks the resulting candidates. Exhaustive, heuristic, and learning-based methods differ in how they generate assignments and use previous evaluations [4, 17, 21], but all reuse the same representation, constraints, and evaluator. The upfront expert effort is paid once and amortized across every candidate the pipeline evaluates.

2.2 Why Reformulation Recurs in Heterogeneous Serving

Consider extending a serving formulation built around one accelerator pool with separate prefill and decode pools [36, 52]. The candidate representation must gain two resource pools, phase-placement decisions, and a KV-cache transfer path. The evaluator must then model queueing in each pool, the transfer cost, and their interaction with end-to-end latency and throughput. Until experts implement the mechanism, an optimizer cannot compare the disaggregated design with designs the original formulation expresses.

A subsequent hardware composition can incur the same work again. Moving attention to PIM [13, 34] or dynamically scheduling operators across NPU and PIM [49] introduces device-side execution, weight-layout, and command-scheduling behavior absent from the preceding formulation. Evaluator coverage makes this second reformulation especially costly because accelerator models [27], serving simulators [1, 8], and distributed-system simulators [38] cover different parts of the resulting system. Candidates beyond this coverage require new models and independent validation.

Once experts add a mechanism and its models, a conventional optimizer can efficiently search its parameters. A new hardware composition, however, may require a different representation and evaluator, causing the same expert work to recur. The bottleneck is repeated reformulation for each new class of designs, not the number of candidates within one formulation.

2.3 What Existing Agentic DSE Leaves Fixed

Existing agentic DSE demonstrates that agents can automate consequential work within a formulation. LLM-DSE optimizes HLS directives within an accelerator parameterization [46], while gem5 Co-Pilot automates parameter search and analysis within gem5 [9]. AgentDSE goes further by using a coding agent to reason from simulator feedback and modify design artifacts across architectural tasks [43]. ArchEval makes these capabilities comparable across simulator-backed architecture challenges [44]. These systems establish the capabilities needed for agent-driven DSE, but the supplied task and verification criterion still determine what counts as a candidate and a successful result. Even when an agent assembles the workflow or modifies design artifacts, exploration remains bounded by that formulation.

3 Agent-Driven Pathfinding

We argue that the agent should not stop at the formulation it is given. When the evidence points to designs outside it, the agent should change the formulation itself instead of waiting for experts to do so. We call this paradigm *agent-driven pathfinding*, in which the agent charts the design space while exploring it. Throughout this section, we illustrate pathfinding with the hypothesis that offloading attention to PIM relieves memory pressure on the NPU [13, 34].

3.1 Executable Candidates

Conventional optimization begins after experts have expressed each candidate as an assignment to predefined parameters. Pathfinding begins one step earlier, when a hypothesis may not yet be expressible in those parameters. The agent makes the hypothesis executable by modifying the relevant system artifacts, such as a mapping, scheduling policy, runtime mechanism, or performance model. A candidate in pathfinding is an executable system modification paired with an evaluator that can measure it.

We illustrate this with the PIM offload hypothesis, starting from a formulation for NPU-based serving that exposes the number of NPUs, the tensor-parallel degree, and the batch size, with a serving simulator and an NPU performance model as its evaluator. No setting of these parameters expresses offloading attention to PIM. Testing it requires an implementation of operator placement, data movement, and device command scheduling, together with an evaluator that captures their effects across the NPU and PIM. Conventional DSE must wait for experts to build both, whereas in pathfinding the agent builds them and leaves an offload threshold and resource allocation that a conventional optimizer can search.

3.2 Diagnostic Evaluation

An executable candidate must produce enough evidence to determine the next design action. Objective and constraint values may suffice to rank assignments within a fixed formulation, but alone they cannot generally reveal whether a poor result comes from an unfortunate parameter setting or an inaccurate model. To tell these apart, the evaluator returns diagnostic artifacts with objective metrics, such as execution traces and cross-fidelity discrepancies.

In the PIM example, evaluation across batch sizes might show that the offload candidate improves performance at small batches but loses its advantage as the batch grows, with execution traces

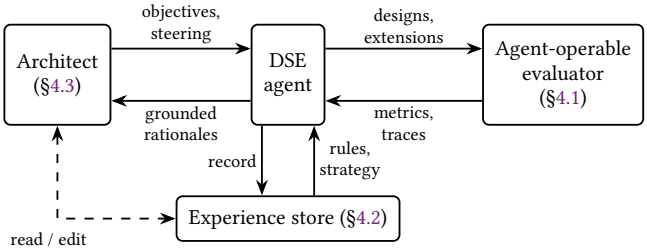


Figure 1: An envisioned agent-driven pathfinding loop. Evaluation grounds an experience store that links hypotheses, evidence, and decisions, while the architect steers subsequent exploration through the same state.

revealing a saturated PIM command queue and an underutilized NPU. The objective value identifies the losing candidate, whereas the trace identifies command scheduling as the limiting interaction. The diagnosis supports two distinct next actions. Exploration can bound the offload with a batch-size-dependent policy, or test whether a new memory scheduler removes the contention.

3.3 Evidence-Driven Reformulation

A diagnosis can call for optimizing within the current formulation or for revising it. The deciding question is whether a choice that the formulation already exposes can remove the limiting behavior. In the PIM offload example, diagnosis attributes the loss at large batches to a saturated PIM command queue. Restricting offload to small batches passes this test, because implementing the offload already exposed an offload threshold that a conventional optimizer can tune. Relieving the queue contention does not, because no exposed choice changes how PIM commands are scheduled.

A new mechanism does not by itself enlarge the formulation. If a new PIM memory scheduler were better at every batch size, it would simply replace the existing one. A scheduler that relieves contention at large batches, however, may add overhead at small ones, making the better choice depend on batch size. The agent then keeps both schedulers as a new structural choice, exposes the parameters and constraints of the new one, and extends the evaluator to model it. The formulation grows only when a new mechanism introduces such a tradeoff.

Each of these decisions trusts the evaluator's model of PIM command scheduling, which produced the saturated queue in the first place. If that queue behavior falls outside the model's validated coverage, or if evaluators at different fidelities disagree, the agent must validate or extend the evaluator before changing the design.

Once the formulation is fixed again, conventional DSE searches within it until diagnosis finds another limit that no exposed choice can remove, so pathfinding acts as an outer loop around conventional DSE rather than a replacement for it. Existing agentic DSE already automates parameter tuning and candidate revision within a supplied formulation (Section 2.3), so what pathfinding adds is the ability to expand the formulation and its evaluator.

4 Requirements for Agent-Driven Pathfinding

The pathfinding loop in Section 3 requires a systems substrate connecting the agent to evaluation, accumulated evidence, and architectural judgment. Figure 1 shows these dependencies as one interlocking loop. We leave agent internals such as model choice and topology aside, since they change too quickly to anchor a systems agenda, and specify only the interfaces the loop needs. We propose three requirements: an evaluator the agent can operate and extend, an experience store that preserves evidence across reformulations, and a control plane through which the architect can inspect and steer the same state the agent uses.

4.1 Agent-Operable Evaluation Infrastructure

Why a fixed evaluator fails. Every empirical conclusion about a candidate must be grounded in evaluation, yet no existing environment provides the coverage, extensibility, and cost-fidelity control required for open-ended heterogeneous DSE. Exploring even the deployable GPU configurations for one model can require tens of thousands of GPU-hours [1], while candidates involving prototype or unbuilt devices offer limited or no direct measurement anchors [24, 35]. Most candidates must be simulated, yet each existing tool models only one slice of accelerator, memory-side compute, storage, network, or serving behavior [27]. Heterogeneous serving proposals typically build or adapt their own evaluators [12, 13, 19, 34], repeating at the evaluation layer the formulation work of Section 2.2.

One interface for operation and extension. Because the agent goes looking for mechanisms the formulation lacks, it will routinely hit behavior the evaluator cannot model. The agent should be able to use and extend the evaluator through one interface, and a minimal version needs three operations. The `evaluate` operation runs a candidate on a workload at a requested fidelity and returns objective metrics with uncertainty, diagnostic traces, constraint violations, and the version of every model involved. For a candidate it cannot model, it instead names the missing behavior and the extension point at which a model, adapter, or measurement path must be attached. The `extend` operation attaches such a model as a new evaluator version marked provisional, and `validate` runs independent checks on that version and records which it has passed (Section 5). Results that depend on a provisional model can steer exploration, but they cannot justify pruning or design conclusions until the model passes validation.

A calibrated fidelity ladder. Evaluation cost determines how often a candidate can be tested, while fidelity determines how strongly its result can support a design decision. The available building blocks span analytical models that evaluate hypothetical hardware in minutes with reported end-to-end errors near 4–6% [5, 50], event-driven serving simulators [1, 8], detailed component simulators [25, 26], and measurements from prototypes or deployed hardware when available [24]. Prior work has already combined analytical and high-fidelity models within one search [40]. A common candidate schema and metric semantics, coupled with cross-fidelity calibration and uncertainty, would let the agent screen many candidates cheaply and promote selected designs to progressively stronger evaluation without losing comparability. Simthesizer [18] further shows that a coding agent can extend a simulator and validate the

extension against vLLM-based deployments [23] with under 3% throughput error. In pathfinding, the search itself would request such extensions, and validating them when no hardware anchor exists is an open problem (Section 5).

4.2 Evidence-Backed Experience and Adaptive Exploration

State that survives reformulation. Fixed-space optimizers already accumulate samples, populations, or learned surrogates to choose later candidates [4, 17, 21, 29]. Their state is indexed by the parameters and objectives of a fixed formulation, so a newly introduced mechanism changes the coordinates for which that state has observations. A score also records how a candidate performed without preserving the bottleneck, parameter interaction, or violated assumption that explains the outcome. The agent instead needs state organized around hypotheses, evidence, and diagnoses, which stay meaningful as the formulation changes.

Experience as a first-class artifact. The experience store separates three typed layers: evaluation entries, distilled rules, and the current search strategy. Each entry binds a hypothesis and a versioned candidate to the evaluator version that measured it, the resulting metrics and traces, and the diagnosis drawn from them. Each rule is a conditional conclusion with an explicit scope, validation status, and links to the entries that support it. The strategy artifact records the axes, ordering, granularity, and evaluation budget that currently govern exploration, together with the evidence that motivated those choices. Agent reflection and episodic memory [41, 45] distill experience into reusable lessons, but those lessons carry no scope or supporting evidence, so nothing tells the agent when they stop applying.

Validated rules can prune redundant candidates within one search loop. Across platforms, retrieved rules serve only as priors until their conditions are checked and their conclusions are revalidated on the new platform. Distilled trajectories can improve later agent performance [47, 51], but transferred rules require revalidation before they can prune candidates.

The strategy must also adapt. Conventional adaptive optimizers can change their sampling distribution within a supplied formulation [16], whereas the agent must also revise which axes and mechanisms the formulation exposes. Accumulated evidence may justify coarsening an insensitive axis, searching interacting knobs jointly, or introducing a new mechanism. Each strategy revision must be versioned with the entries that motivated it so the architect can audit or override both the decision and its scope. Feedback-conditioned proposal methods return previous candidates and scores to the model to improve later proposals [30, 39, 48], but the space they propose into stays fixed.

4.3 Human-in-the-Loop Pathfinding

Revisable design intent. Because the agent can change the formulation itself, no specification written at launch can anticipate every objective or constraint that later exploration will need. The architect must be able to revise objectives, add scoped constraints, and inject architectural priors while the search runs. The architect keeps authority over design intent, and the agent does the implementation, evaluation, and revision [7].

Traceable evidence. When the formulation and evaluator behind a recommendation have both changed during the search, the architect needs to inspect its evidence and assumptions, not only the final design. An explanation generated after the decision is insufficient because model rationales can differ from the factors that actually caused their outputs [42]. The architect should instead be able to trace every proposed mechanism, pruned region, and strategy revision to the relevant experience entries, evaluator versions, and diagnostic artifacts.

Review that scales with revisions. An agent can evaluate far more candidates than an architect can inspect, so reviewing them one by one would make the architect the bottleneck that pathfinding aims to remove. Most candidates, however, are points in a formulation that the architect has already approved. What needs review is the small set of decisions that change what the loop can explore or conclude: expanding the formulation, promoting an evaluator extension, granting a rule the authority to prune, and recommending a design. There are only as many such decisions as there are revisions, a small fraction of the candidates evaluated. A pending review blocks only what it would authorize, so search within the approved formulation continues in the meantime.

A shared control plane. The experience store provides the shared state through which the architect specifies, inspects, and intervenes in exploration. Hard constraints, architectural priors, and distilled rules remain distinct typed records, so a tentative heuristic cannot silently acquire the authority of a power limit. The architect can invalidate a rule, pin an axis, or force exploration of a deprioritized region. Each intervention is recorded and affects subsequent exploration through the same state the agent consumes, without requiring the search to restart.

5 Open Problems

Section 4 makes evaluator use and extension part of exploration, so evaluator validity can no longer be treated as an external precondition. Search can exploit error in the existing evaluator, and an evaluator extension can introduce new error correlated with the candidate it assesses.

Search amplifies evaluator error. When evaluation is approximate, an optimizer selects candidates by predicted rather than true performance, and repeated selection favors candidates whose predictions benefit from model error. The apparently best designs may therefore be precisely those that require the strongest validation. Guarding against this requires uncertainty estimates on every metric, calibration against real hardware where it exists, and re-evaluation of favored regions at higher fidelity before they support design conclusions. The open question is how to allocate validation across fidelities while keeping low-fidelity exploration cheap.

Evaluator extensions require independent validation. Extending the evaluator can also introduce new error. An agent-written extension may encode the same misconception as the candidate it is meant to evaluate, so agreement between the two corroborates nothing. Simthesizer validates synthesized extensions against deployed systems [18], but candidate hardware often has no deployed counterpart. Validation must then combine independent checks such as cross-fidelity agreement, invariants over operations and bytes moved, held-out measurements, and small prototypes, and

record which checks each extension has passed. The unresolved problem is deciding when that evidence is sufficient for the extension to justify pruning or a design recommendation.

Measuring progress. Existing agentic DSE benchmarks such as ArchEval [44] supply the formulation and the verification criterion, so they measure only search within a formulation. A pathfinding benchmark should instead begin from a formulation that omits a mechanism known to matter, as a monolithic serving formulation omits phase disaggregation. Its primary metric is the validated gain over the best design that the initial formulation can express, which an agent confined to that formulation can at best match. The agent time, simulation time, and architect interventions spent to reach that gain measure its cost, and the gap between predicted and validated performance shows how far the gain can be trusted. Because agents may recall such well-known mechanisms from their training data, benchmarks should favor mechanisms published after the agent's training cutoff.

6 Conclusion

Heterogeneous LLM serving weakens the fixed-formulation amortization on which conventional DSE relies, turning the repeated work of expressing and evaluating new mechanisms into the bottleneck. Agent-driven pathfinding brings this work into an outer loop that constructs executable candidates, diagnoses their behavior from evidence, and revises the formulation when necessary. Realizing this paradigm requires agent-operable evaluation, evidence-backed experience, and architect control over the state that drives exploration. The immediate task is to keep the evolving evaluator trustworthy by preventing search from amplifying model error and requiring independent validation for every extension.

References

[1] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. Vidur: A Large-Scale Simulation Framework for LLM Inference. In *MLSys*.

[2] Anthropic. 2025. Claude 3.7 Sonnet and Claude Code. https://www.anthropic.com/news/claude-3-7-sonnet.

[3] Anthropic. 2025. Expanding Our Use of Google Cloud TPUs and Services. https://anthropic.com/news/expanding-our-use-of-google-cloud-tpus-and-services.

[4] Chen Bai, Qi Sun, Jianwang Zhai, Yuzhe Ma, Bei Yu, and Martin D. F. Wong. 2021. BOOM-Explorer: RISC-V BOOM Microarchitecture Design Space Exploration Framework. In *ICCAD*.

[5] Abhimanyu Bambhaniya, Ritik Raj, Geonhwa Jeong, Souvik Kundu, Sudarshan Srinivasan, Suvinay Subramanian, Midhilesh Elavazhagan, Madhu Kumar, and Tushar Krishna. 2024. Demystifying AI Platform Design for Distributed Inference of Next-Generation LLM Models. arXiv preprint arXiv:2406.01698.

[6] Hongzheng Chen, Alexander Novikov, Ngân Vũ, Hanna Alam, Zhiru Zhang, Aiden Grossman, Mircea Trofin, and Amir Yazdanbakhsh. 2026. Magellan: Autonomous Discovery of Novel Compiler Optimization Heuristics with AlphaEvolve. arXiv preprint arXiv:2601.21096.

[7] Audrey Cheng, Shu Liu, Melissa Pan, Zhifei Li, Bowen Wang, Alex Krentsel, Tian Xia, Mert Cemri, Jongseok Park, Shuo Yang, Jeff Chen, Lakshya Agrawal, Aditya Desai, Jiarong Xing, Koushik Sen, Matei Zaharia, and Ion Stoica. 2025. Barbarians at the Gate: How AI is Upending Systems Research. arXiv preprint arXiv:2510.06189.

[8] Jaehong Cho, Minsu Kim, Hyunmin Choi, Guseul Heo, and Jongse Park. 2024. LLMServingSim: A HW/SW Co-Simulation Infrastructure for LLM Inference Serving at Scale. In *IISWC*.

[9] Zuoming Fu, Alex Manley, and Mohammad Alian. 2025. gem5 Co-Pilot: AI Assistant Agent for Architectural Design Space Exploration. arXiv preprint arXiv:2510.19577.

[10] Gartner. 2026. Gartner Forecasts Worldwide AI-Optimized IaaS Spending to Grow 96% in 2026. Gartner Press Release, https://www.gartner.com/en/newsroom/press-releases/2026-08-10-gartner-forecasts-worldwide-artificial-intelligence-optimized-iaas-spending-to-grow-96-percent-in-2026.

[11] Daya Guo et al. 2025. DeepSeek-R1 Incentivizes Reasoning in LLMs through Reinforcement Learning. *Nature* 645 (2025).

[12] Yintao He, Haiyu Mao, Christina Giannoula, Mohammad Sadrosadati, Juan Gómez-Luna, Huawei Li, Xiaowei Li, Ying Wang, and Onur Mutlu. 2025. PAPI: Exploiting Dynamic Parallelism in Large Language Model Decoding with a Processing-In-Memory-Enabled Computing System. In *ASPLOS*.

[13] Guseul Heo, Sangyeop Lee, Jaehong Cho, Hyunmin Choi, Sanghyeon Lee, Hyungkyu Ham, Gwangsun Kim, Divya Mahajan, and Jongse Park. 2024. NeuPIMs: NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing. In *ASPLOS*.

[14] Youhe Jiang, Ran Yan, Xiaozhe Yao, Yang Zhou, Beidi Chen, and Binhang Yuan. 2024. HexGen: Generative Inference of Large Language Model over Heterogeneous Environment. In *ICML*.

[15] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *ICLR*.

[16] Sheng-Chun Kao, Geonhwa Jeong, and Tushar Krishna. 2020. ConfuciuX: Autonomous Hardware Resource Assignment for DNN Accelerators Using Reinforcement Learning. In *MICRO*.

[17] Sheng-Chun Kao and Tushar Krishna. 2020. GAMMA: Automating the HW Mapping of DNN Models on Accelerators via Genetic Algorithm. In *ICCAD*.

[18] Wonung Kim, Hyunmin Choi, Minsu Kim, Jaehong Cho, Yeongwook Kim, and Jongse Park. 2026. Simthesizer: An Agent-Driven Simulation Framework for LLM Serving Systems. arXiv preprint arXiv:2608.24650.

[19] Wonung Kim, Yubin Lee, Yoonsung Kim, Jinwoo Hwang, Seongryong Oh, Jiyong Jung, Aziz Huseynov, Woong Gyu Park, Chang Hyun Park, Divya Mahajan, and Jongse Park. 2025. Pimba: A Processing-in-Memory Acceleration for Post-Transformer Large Language Model Serving. In *MICRO*.

[20] Srivatsan Krishnan, Amir Yazdanbakhsh, Shvetank Prakash, Jason Jabbour, Ikechukwu Uchendu, Susobhan Ghosh, Behzad Boroujerdian, Daniel Richins, Devashree Tripathy, Aleksandra Faust, and Vijay Janapa Reddi. 2023. ArchGym: An Open-Source Gymnasium for Machine Learning Assisted Architecture Design. In *ISCA*.

[21] Aviral Kumar, Amir Yazdanbakhsh, Milad Hashemi, Kevin Swersky, and Sergey Levine. 2022. Data-Driven Offline Optimization for Architecting Hardware Accelerators. In *ICLR*.

[22] Hyoukjun Kwon, Prasanth Chatarasi, Michael Pellauer, Angshuman Parashar, Vivek Sarkar, and Tushar Krishna. 2019. Understanding Reuse, Performance, and Hardware Cost of DNN Dataflows: A Data-Centric Approach. In *MICRO*.

[23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP*.

[24] Sukhan Lee, Shinhaeng Kang, Jaehoon Lee, Hyeonsu Kim, Eojin Lee, Seungwoo Seo, Hosang Yoon, Seungwon Lee, Kyounghwan Lim, Hyunsung Shin, Jinhyun Kim, Seongil O, Anand Iyer, David Wang, Kyomin Sohn, and Nam Sung Kim. 2021. Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology: Industrial Product. In *ISCA*.

[25] Shang Li, Zhiyuan Yang, Dhiraj Reddy, Ankur Srivastava, and Bruce Jacob. 2020. DRAMsim3: A Cycle-Accurate, Thermal-Capable DRAM Simulator. *IEEE Computer Architecture Letters* 19, 2 (2020), 106–109.

[26] Haocong Luo, Ataberk Olgun, A. Giray Yağlıkçı, Yahya Can Tuğrul, Steve Rhyner, Meryem Banu Cavlak, Joël Lindegger, Mohammad Sadrosadati, and Onur Mutlu. 2024. Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator. *IEEE Computer Architecture Letters* 23, 1 (2024), 112–116.

[27] Stefan Maczynski, Amlan Ganguly, and Mark Indovina. 2025. The State of Simulation Frameworks for Evaluating Emerging LLM Accelerators. In *GLSVLSI*.

[28] Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang, Zhihao Jia, and Rashmi Vinayak. 2025. Helix: Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow. In *ASPLOS*.

[29] Luigi Nardi, David Koeplinger, and Kunle Olukotun. 2019. Practical Design Space Exploration. In *MASCOTS*.

[30] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. 2025. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. arXiv preprint arXiv:2506.13131.

[31] OpenAI. 2024. Learning to Reason with LLMs. https://openai.com/index/learning-to-reason-with-llms/.

[32] OpenAI. 2025. Introducing Codex. https://openai.com/index/introducing-codex/.

[33] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A. Ying, Anurag Mukkara, Rangharajan Venkatesan, Brucek Khailany, Stephen W. Keckler, and Joel Emer. 2019. Timeloop: A Systematic Approach to DNN Accelerator Evaluation. In *ISPASS*.

[34] Jaehyun Park, Jaewan Choi, Kwanhee Kyung, Michael Jaemin Kim, Yongsuk Kwon, Nam Sung Kim, and Jung Ho Ahn. 2024. AttAcc! Unleashing the Power of PIM for Batched Transformer-based Generative Model Inference. In *ASPLOS*.

[35] Sangsoo Park, KyungSoo Kim, Jinin So, Jin Jung, Jonggeon Lee, Kyoungwan Woo, Nayeon Kim, Younghyun Lee, Hyungyo Kim, Yongsuk Kwon, Jinhyun Kim, Jieun Lee, YeonGon Cho, Yongmin Tai, Jeonghyeon Cho, Hoyoung Song, Jung Ho Ahn, and Nam Sung Kim. 2024. An LPDDR-based CXL-PNM Platform for TCO-efficient Inference of Transformer-based Large Language Models. In *HPCA*.

[36] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *ISCA*.

[37] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *FAST*.

[38] Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. 2020. ASTRA-SIM: Enabling SW/HW Co-Design Exploration for Distributed DL Training Platforms. In *ISPASS*.

[39] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024. Mathematical Discoveries from Program Search with Large Language Models. *Nature* (2024).

[40] Chirag Sakhuja, Charles Hong, and Calvin Lin. 2024. Polaris: Multi-Fidelity Design Space Exploration of Deep Learning Accelerators. arXiv preprint arXiv:2412.15548.

[41] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *NeurIPS*.

[42] Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. 2023. Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting. In *NeurIPS*.

[43] Chenyu Wang, Jiahe Caroline Shi, David Kong, Duane S. Boning, Zishen Wan, Yilun Du, and Vijay Janapa Reddi. 2026. AgentDSE: Reasoning-Augmented Architectural Design Space Exploration. arXiv preprint arXiv:2606.21836.

[44] Chenyu Wang, Zishen Wan, Jeffrey Ma, Shvetank Prakash, Zhenting Qi, Haebin Do, Andy Cheng, Arya Tschand, Jiahe Shi, Yilun Du, and Vijay Janapa Reddi. 2026. ArchEval: Measuring AI Agents as Computer Architects. arXiv preprint arXiv:2607.03601.

[45] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research* (2024).

[46] Hanyu Wang, Xinrui Wu, Zijian Ding, Su Zheng, Chengyue Wang, Neha Prakriya, Tony Nowatzki, Yizhou Sun, and Jason Cong. 2025. LLM-DSE: Searching Accelerator Parameters with LLM Agents. arXiv preprint arXiv:2505.12188.

[47] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent Workflow Memory. In *ICML*.

[48] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large Language Models as Optimizers. In *ICLR*.

[49] Jiaqi Yang, Jiayi Li, Yihan Fu, Hongxiao Zhao, Zhan Chen, Qiuping Wu, Yuchao Yang, and Bonan Yan. 2026. Beyond Prefill-Decode Disaggregation: Dissecting LLM Inference for Heterogeneous Platforms via Dynamic Operator Scheduling. arXiv preprint arXiv:2607.25498.

[50] Hengrui Zhang, August Ning, Rohan Baskar Prabhakar, and David Wentzlaff. 2024. LLMCompass: Enabling Efficient Hardware Design for Large Language Model Inference. In *ISCA*.

[51] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM Agents Are Experiential Learners. In *AAAI*.

[52] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *OSDI*.

[53] Ruidong Zhu, Ziheng Jiang, Chao Jin, Peng Wu, Cesar A. Stuardo, Dongyang Wang, Xinlei Zhang, Huaping Zhou, Haoran Wei, Yang Cheng, Jianzhe Xiao, Xinyi Zhang, Lingjun Liu, Haibin Lin, Li-Wen Chang, Jianxi Ye, Xiao Yu, Xuanzhe Liu, Xin Jin, and Xin Liu. 2025. MegaScale-Infer: Efficient Mixture-of-Experts Model Serving with Disaggregated Expert Parallelism. In *SIGCOMM*.